

Inheritance

1. Create a class called Person.

Populate the class with below private fields

- 1. FirstName as string**
- 2. LastName as string**
- 3. EmailAddress as string**
- 4. DateOfBirth as DateTime**

Add a constructor that accept FirstName, LastName, Email, DateOfBirth as parameters

Add read-only properties that return the following computed information:

- 1. IsAdult as bool - whether or not the person is over 18**
- 2. SunSign as string - the traditional western sun sign of the person**
- 3. IsBirthDay as bool - whether or not today is the person's birthday**
- 4. ScreenName as string- a default screen name that you might see being offered to a first time user of Gmail or Yahoo (e.g. Hari Joe born on May 25th, 1980 might be hdoe525 or haridoe052580)**

Create a class called Employee which should be derived from Person class. Employee class should add the following properties:

- 1. Salary as double**

In the Main method create object of Employee class and test the behaviour by calling read-only properties

```
using System;
```

```
class Person
```

```
{
```

```
    // Private fields
```

```
    private string FirstName;
```

```
    private string LastName;
```

```
    private string EmailAddress;
```

```
    private DateTime DateOfBirth;
```

```
    // Constructor
```

```
    public Person(string firstName, string lastName, string email,  
DateTime dob)
```

```
    {
```

```
        FirstName = firstName;
```

```
        LastName = lastName;
```

```
        EmailAddress = email;
```

```
        DateOfBirth = dob;
```

```
    }
```

```
    // Read-only property - Is Adult
```

```
    public bool IsAdult
```

```
    {
```

```

get
{
    int age = DateTime.Today.Year - DateOfBirth.Year;
    if (DateOfBirth.Date > DateTime.Today.AddYears(-age))
        age--;

    return age >= 18;
}

```

```

// Read-only property - Sun Sign
public string SunSign
{
    get
    {
        int day = DateOfBirth.Day;
        int month = DateOfBirth.Month;

        if ((month == 3 && day >= 21) || (month == 4 && day <= 19))
            return "Aries";
        else if ((month == 4 && day >= 20) || (month == 5 && day <=
20))
            return "Taurus";
    }
}

```

```
    else if ((month == 5 && day >= 21) || (month == 6 && day <=
20))
        return "Gemini";
    else if ((month == 6 && day >= 21) || (month == 7 && day <=
22))
        return "Cancer";
    else if ((month == 7 && day >= 23) || (month == 8 && day <=
22))
        return "Leo";
    else if ((month == 8 && day >= 23) || (month == 9 && day <=
22))
        return "Virgo";
    else if ((month == 9 && day >= 23) || (month == 10 && day <=
22))
        return "Libra";
    else if ((month == 10 && day >= 23) || (month == 11 && day
<= 21))
        return "Scorpio";
    else if ((month == 11 && day >= 22) || (month == 12 && day
<= 21))
        return "Sagittarius";
    else if ((month == 12 && day >= 22) || (month == 1 && day <=
19))
        return "Capricorn";
```

```
        else if ((month == 1 && day >= 20) || (month == 2 && day <=
18))
            return "Aquarius";
        else
            return "Pisces";
    }
}
```

// Read-only property - Birthday

public bool IsBirthDay

```
{
    get
    {
        return (DateTime.Today.Month == DateOfBirth.Month &&
            DateTime.Today.Day == DateOfBirth.Day);
    }
}
```

// Read-only property - Screen Name

public string ScreenName

```
{
    get
    {
```

```

        return FirstName.ToLower() +
            LastName.ToLower() +
            DateOfBirth.ToString("MMddyy");
    }
}

// Read-only properties to display details
public string FullName
{
    get { return FirstName + " " + LastName; }
}

public string Email
{
    get { return EmailAddress; }
}
}

// Employee class
class Employee : Person
{
    public double Salary { get; set; }
}

```

```

    public Employee(string firstName, string lastName, string email,
DateTime dob, double salary)
        : base(firstName, lastName, email, dob)
    {
        Salary = salary;
    }
}

```

// Main class

```
class Program
```

```

{
    static void Main(string[] args)
    {
        Employee emp = new Employee(
            "Hari",
            "Doe",
            "hari@gmail.com",
            new DateTime(2000, 5, 25),
            50000);

        Console.WriteLine("Employee Details");
        Console.WriteLine("-----");
        Console.WriteLine("Name      : " + emp.FullName);
    }
}

```

```
        Console.WriteLine("Email    : " + emp.Email);  
        Console.WriteLine("Salary   : " + emp.Salary);  
        Console.WriteLine("Is Adult : " + emp.IsAdult);  
        Console.WriteLine("Sun Sign : " + emp.SunSign);  
        Console.WriteLine("Birthday : " + emp.IsBirthDay);  
        Console.WriteLine("ScreenName : " + emp.ScreenName);  
    }  
}
```

OUTPUT:

Employee Details

Name : Hari Doe

Email : hari@gmail.com

Salary : 50000

Is Adult : True

Sun Sign : Gemini

Birthday : False

ScreenName : haridoe052500

**2.create a derived class HourlyEmployee , and
PermanentEmployee from the above**

Person class which will have the below properties respectively.

HourlyEmployee

1) HoursWorked as double

2) PayPerHour as double

PermanantEmployee

1) HRA as double

2) DA as double

3) Tax as double

4) NetPay as double

5) TotalPay as double

create an instance in Main method and test the classes

using System;

class Person

{

// Private fields

private string FirstName;

private string LastName;

private string EmailAddress;

private DateTime DateOfBirth;

// Constructor

public Person(string firstName, string lastName, string email,
DateTime dob)

{

FirstName = firstName;

LastName = lastName;

EmailAddress = email;

```
    DateOfBirth = dob;  
}
```

```
// Read-only property
```

```
public bool IsAdult
```

```
{  
    get  
    {  
        int age = DateTime.Today.Year - DateOfBirth.Year;  
        if (DateOfBirth > DateTime.Today.AddYears(-age))  
            age--;  
        return age >= 18;  
    }  
}
```

```
// Read-only property
```

```
public bool IsBirthDay
```

```
{  
    get  
    {  
        return DateTime.Today.Month == DateOfBirth.Month &&  
            DateTime.Today.Day == DateOfBirth.Day;  
    }  
}
```

```
}
```

```
// Read-only property
```

```
public string ScreenName
```

```
{
```

```
    get
```

```
    {
```

```
        return FirstName.ToLower() +
```

```
            LastName.ToLower() +
```

```
            DateOfBirth.ToString("MMddyy");
```

```
    }
```

```
}
```

```
// Read-only property
```

```
public string SunSign
```

```
{
```

```
    get
```

```
    {
```

```
        int day = DateOfBirth.Day;
```

```
        int month = DateOfBirth.Month;
```

```
        if ((month == 3 && day >= 21) || (month == 4 && day <= 19))
```

```
            return "Aries";
```

```
    else if ((month == 4 && day >= 20) || (month == 5 && day <=
20))
        return "Taurus";
    else if ((month == 5 && day >= 21) || (month == 6 && day <=
20))
        return "Gemini";
    else if ((month == 6 && day >= 21) || (month == 7 && day <=
22))
        return "Cancer";
    else if ((month == 7 && day >= 23) || (month == 8 && day <=
22))
        return "Leo";
    else if ((month == 8 && day >= 23) || (month == 9 && day <=
22))
        return "Virgo";
    else if ((month == 9 && day >= 23) || (month == 10 && day <=
22))
        return "Libra";
    else if ((month == 10 && day >= 23) || (month == 11 && day
<= 21))
        return "Scorpio";
    else if ((month == 11 && day >= 22) || (month == 12 && day
<= 21))
        return "Sagittarius";
```

```

        else if ((month == 12 && day >= 22) || (month == 1 && day <=
19))
            return "Capricorn";
        else if ((month == 1 && day >= 20) || (month == 2 && day <=
18))
            return "Aquarius";
        else
            return "Pisces";
    }
}

```

```

public string FullName
{
    get { return FirstName + " " + LastName; }
}

public string Email
{
    get { return EmailAddress; }
}
}

```

```

// HourlyEmployee Class
class HourlyEmployee : Person

```

```

{
    public double HoursWorked { get; set; }
    public double PayPerHour { get; set; }

    public HourlyEmployee(string fname, string lname, string email,
DateTime dob,
        double hoursWorked, double payPerHour)
        : base(fname, lname, email, dob)
    {
        HoursWorked = hoursWorked;
        PayPerHour = payPerHour;
    }

    public double Salary
    {
        get { return HoursWorked * PayPerHour; }
    }
}

// PermanentEmployee Class
class PermanentEmployee : Person
{
    public double BasicPay { get; set; }
    public double HRA { get; set; }
}

```

```

    public double DA { get; set; }

    public double Tax { get; set; }

    public PermanentEmployee(string fname, string lname, string
email, DateTime dob,

                                double basicPay, double hra, double da, double tax)

        : base(fname, lname, email, dob)
    {
        BasicPay = basicPay;

        HRA = hra;

        DA = da;

        Tax = tax;
    }

    public double TotalPay
    {
        get { return BasicPay + HRA + DA; }
    }

    public double NetPay
    {
        get { return TotalPay - Tax; }
    }
}

```

```
// Main Class
```

```
class Program
{
    static void Main(string[] args)
    {
        // Hourly Employee
        HourlyEmployee h = new HourlyEmployee(
            "Rahul",
            "Kumar",
            "rahul@gmail.com",
            new DateTime(2001, 6, 15),
            40,
            500);

        Console.WriteLine("Hourly Employee");
        Console.WriteLine("-----");
        Console.WriteLine("Name      : " + h.FullName);
        Console.WriteLine("Email     : " + h.Email);
        Console.WriteLine("Hours Worked : " + h.HoursWorked);
        Console.WriteLine("Pay Per Hour : " + h.PayPerHour);
        Console.WriteLine("Salary     : " + h.Salary);
        Console.WriteLine("Is Adult   : " + h.IsAdult);
        Console.WriteLine("Sun Sign   : " + h.SunSign);
        Console.WriteLine("Birthday   : " + h.IsBirthDay);
    }
}
```

```
Console.WriteLine("Screen Name : " + h.ScreenName);

Console.WriteLine();

// Permanent Employee

PermanentEmployee p = new PermanentEmployee(

    "Sita",

    "Reddy",

    "sita@gmail.com",

    new DateTime(1998, 9, 20),

    30000,

    5000,

    3000,

    2000);


Console.WriteLine("Permanent Employee");

Console.WriteLine("-----");

Console.WriteLine("Name      : " + p.FullName);

Console.WriteLine("Email      : " + p.Email);

Console.WriteLine("Basic Pay   : " + p.BasicPay);

Console.WriteLine("HRA        : " + p.HRA);

Console.WriteLine("DA         : " + p.DA);

Console.WriteLine("Tax        : " + p.Tax);

Console.WriteLine("Total Pay   : " + p.TotalPay);

Console.WriteLine("Net Pay     : " + p.NetPay);
```

```
        Console.WriteLine("Is Adult    : " + p.IsAdult);
        Console.WriteLine("Sun Sign    : " + p.SunSign);
        Console.WriteLine("Birthday    : " + p.IsBirthDay);
        Console.WriteLine("Screen Name : " + p.ScreenName);
    }
}
```

OUTPUT:

Hourly Employee

Name : Rahul Kumar

Email : rahul@gmail.com

Hours Worked : 40

Pay Per Hour : 500

Salary : 20000

Is Adult : True

Sun Sign : Gemini

Birthday : False

Screen Name : rahulkumar061501

Permanent Employee

Name : Sita Reddy

Email : sita@gmail.com

Basic Pay : 30000

HRA : 5000

DA : 3000

Tax : 2000

Total Pay : 38000

Net Pay : 36000

Is Adult : True

Sun Sign : Virgo

Birthday : False

Screen Name : sitareddy092098

3. Create an interface called IPayable and implement the interface in the above classes(HourlyEmployee & PermanantEmployee).

using System;

interface IPayable

```
{  
    double CalculatePay();  
}
```

class Person

```
{
```

```
private string FirstName;

private string LastName;

private string EmailAddress;

private DateTime DateOfBirth;


    public Person(string firstName, string lastName, string email,
DateTime dob)
    {
        FirstName = firstName;
        LastName = lastName;
        EmailAddress = email;
        DateOfBirth = dob;
    }


    public string FullName
    {
        get { return FirstName + " " + LastName; }
    }


    public string Email
    {
        get { return EmailAddress; }
    }
```

```
}
```

```
// HourlyEmployee Class
```

```
class HourlyEmployee : Person, IPayable
```

```
{
```

```
    public double HoursWorked { get; set; }
```

```
    public double PayPerHour { get; set; }
```

```
    public HourlyEmployee(string fname, string lname, string email,  
                           DateTime dob, double hoursWorked, double  
payPerHour)
```

```
        : base(fname, lname, email, dob)
```

```
{
```

```
    HoursWorked = hoursWorked;
```

```
    PayPerHour = payPerHour;
```

```
}
```

```
    public double CalculatePay()
```

```
{
```

```
        return HoursWorked * PayPerHour;
```

```
}
```

```
}
```

```
// PermanentEmployee Class
class PermanentEmployee : Person, IPayable
{
    public double BasicPay { get; set; }
    public double HRA { get; set; }
    public double DA { get; set; }
    public double Tax { get; set; }

    public PermanentEmployee(string fname, string lname, string
email,
                                DateTime dob, double basicPay,
                                double hra, double da, double tax)
        : base(fname, lname, email, dob)
    {
        BasicPay = basicPay;
        HRA = hra;
        DA = da;
        Tax = tax;
    }

    public double TotalPay
    {
        get { return BasicPay + HRA + DA; }
    }
}
```

```
public double NetPay
{
    get { return TotalPay - Tax; }
}

public double CalculatePay()
{
    return NetPay;
}
}

class Program
{
    static void Main(string[] args)
    {
        HourlyEmployee h = new HourlyEmployee(
            "Rahul", "Kumar", "rahul@gmail.com",
            new DateTime(2001, 6, 15),
            40, 500);

        PermanentEmployee p = new PermanentEmployee(
            "Sita", "Reddy", "sita@gmail.com",
            new DateTime(1998, 9, 20),
            30000, 5000, 3000, 2000);

        Console.WriteLine("Hourly Employee");
```

```
        Console.WriteLine("Name : " + h.FullName);  
        Console.WriteLine("Pay : " + h.CalculatePay());  
  
        Console.WriteLine();  
  
        Console.WriteLine("Permanent Employee");  
        Console.WriteLine("Name : " + p.FullName);  
        Console.WriteLine("Total Pay : " + p.TotalPay);  
        Console.WriteLine("Net Pay  : " + p.CalculatePay());  
    }  
}
```

OUTPUT:

Hourly Employee

Name : Rahul Kumar

Pay : 20000

Permanent Employee

Name : Sita Reddy

Total Pay : 38000

Net Pay : 36000

4. Declare a Method CalculatePay in the interface

using System;

```
// Interface
```

```
interface IPayable
```

```
{
```

```
    double CalculatePay();
```

```
}
```

```
// Person Class
```

```
class Person
```

```
{
```

```
    private string FirstName;
```

```
    private string LastName;
```

```
    private string EmailAddress;
```

```
    private DateTime DateOfBirth;
```

```
    public Person(string firstName, string lastName, string email,  
DateTime dob)
```

```
{
```

```
    FirstName = firstName;
```

```
    LastName = lastName;
```

```
    EmailAddress = email;
```

```
    DateOfBirth = dob;
```

```
}
```

```

    public string FullName
    {
        get { return FirstName + " " + LastName; }
    }

    public string Email
    {
        get { return EmailAddress; }
    }
}

// HourlyEmployee Class
class HourlyEmployee : Person, IPayable
{
    public double HoursWorked { get; set; }
    public double PayPerHour { get; set; }

    public HourlyEmployee(string firstName, string lastName, string
email,
                        DateTime dob, double hoursWorked, double
payPerHour)
        : base(firstName, lastName, email, dob)
    {

```

```

        HoursWorked = hoursWorked;
        PayPerHour = payPerHour;
    }

    public double CalculatePay()
    {
        return HoursWorked * PayPerHour;
    }
}

// PermanentEmployee Class
class PermanentEmployee : Person, IPayable
{
    public double BasicPay { get; set; }
    public double HRA { get; set; }
    public double DA { get; set; }
    public double Tax { get; set; }

    public PermanentEmployee(string firstName, string lastName,
string email,
        DateTime dob, double basicPay,
        double hra, double da, double tax)
        : base(firstName, lastName, email, dob)

```

```
{  
    BasicPay = basicPay;  
    HRA = hra;  
    DA = da;  
    Tax = tax;  
}  
  
public double TotalPay  
{  
    get { return BasicPay + HRA + DA; }  
}  
  
public double NetPay  
{  
    get { return TotalPay - Tax; }  
}  
  
public double CalculatePay()  
{  
    return NetPay;  
}  
}
```

```
// Main Class
class Program
{
    static void Main(string[] args)
    {
        // Hourly Employee Object
        HourlyEmployee h = new HourlyEmployee(
            "Rahul",
            "Kumar",
            "rahul@gmail.com",
            new DateTime(2001, 6, 15),
            40,
            500);

        // Permanent Employee Object
        PermanentEmployee p = new PermanentEmployee(
            "Sita",
            "Reddy",
            "sita@gmail.com",
            new DateTime(1998, 9, 20),
            30000,
            5000,
            3000,
```

2000);

```
Console.WriteLine("Hourly Employee Details");
Console.WriteLine("-----");
Console.WriteLine("Name      : " + h.FullName);
Console.WriteLine("Email      : " + h.Email);
Console.WriteLine("Hours Worked : " + h.HoursWorked);
Console.WriteLine("Pay Per Hour : " + h.PayPerHour);
Console.WriteLine("Salary      : " + h.CalculatePay());

Console.WriteLine();
```

```
Console.WriteLine("Permanent Employee Details");
Console.WriteLine("-----");
Console.WriteLine("Name      : " + p.FullName);
Console.WriteLine("Email      : " + p.Email);
Console.WriteLine("Basic Pay   : " + p.BasicPay);
Console.WriteLine("HRA        : " + p.HRA);
Console.WriteLine("DA         : " + p.DA);
Console.WriteLine("Tax        : " + p.Tax);
Console.WriteLine("Total Pay   : " + p.TotalPay);
Console.WriteLine("Net Pay     : " + p.CalculatePay());
```

```
        Console.ReadKey();  
    }  
}
```

OUTPUT:

Hourly Employee Details

Name : Rahul Kumar
Email : rahul@gmail.com
Hours Worked : 40
Pay Per Hour : 500
Salary : 20000

Permanent Employee Details

Name : Sita Reddy
Email : sita@gmail.com
Basic Pay : 30000
HRA : 5000
DA : 3000
Tax : 2000
Total Pay : 38000
Net Pay : 36000

5.Implement the interface in above derieved classes and test the interface implementation by passing some values

using System;

// Interface

interface IPayable

```
{  
    double CalculatePay();  
}
```

// Base Class

class Person

```
{  
    public string FirstName;  
    public string LastName;  
    public string Email;
```

```
    public Person(string fname, string lname, string email)
```

```
{  
    FirstName = fname;  
    LastName = lname;  
    Email = email;
```

```

    }
}

// Derived Class - HourlyEmployee
class HourlyEmployee : Person, IPayable
{
    public double HoursWorked;
    public double PayPerHour;

    public HourlyEmployee(string fname, string lname, string email,
                           double hoursWorked, double payPerHour)
        : base(fname, lname, email)
    {
        HoursWorked = hoursWorked;
        PayPerHour = payPerHour;
    }

    public double CalculatePay()
    {
        return HoursWorked * PayPerHour;
    }
}

```

```
// Derived Class - PermanentEmployee
class PermanentEmployee : Person, IPayable
{
    public double BasicPay;
    public double HRA;
    public double DA;
    public double Tax;

    public PermanentEmployee(string fname, string lname, string
email,
                                double basicPay, double hra,
                                double da, double tax)
        : base(fname, lname, email)
    {
        BasicPay = basicPay;
        HRA = hra;
        DA = da;
        Tax = tax;
    }

    public double CalculatePay()
    {
        return (BasicPay + HRA + DA) - Tax;
    }
}
```

```
}  
}
```

```
// Main Class
```

```
class Program
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

```
        // Interface reference for HourlyEmployee
```

```
        IPayable h = new HourlyEmployee(
```

```
            "Rahul",
```

```
            "Kumar",
```

```
            "rahul@gmail.com",
```

```
            40,
```

```
            500);
```

```
        // Interface reference for PermanentEmployee
```

```
        IPayable p = new PermanentEmployee(
```

```
            "Sita",
```

```
            "Reddy",
```

```
            "sita@gmail.com",
```

```
            30000,
```

```
            5000,
```

```
3000,  
2000);
```

```
Console.WriteLine("Hourly Employee Pay = " + h.CalculatePay());  
Console.WriteLine("Permanent Employee Pay = " +  
p.CalculatePay());
```

```
Console.ReadKey();  
}  
}
```

OUTPUT:

Hourly Employee Pay = 20000

Permanent Employee Pay = 36000

POLYMORPHISM

Write a Program to Implement the Compile time polymorphism using Function Overloading approach.

1. Calculate the area of circle, triangle and rectangle using function overloading:

using System;

```
class Area
{
    public double CalculateArea(double radius)
    {
        return 3.14 * radius * radius;
    }

    public double CalculateArea(double length, double breadth)
    {
        return length * breadth;
    }

    public double CalculateArea(float baseValue, float height)
    {
        return 0.5 * baseValue * height;
    }
}

class Program
{
    static void Main()
```

```
{  
    Area obj = new Area();  
  
    Console.Write("Enter Radius of Circle: ");  
    double r = Convert.ToDouble(Console.ReadLine());  
  
    Console.Write("Enter Length of Rectangle: ");  
    double l = Convert.ToDouble(Console.ReadLine());  
  
    Console.Write("Enter Breadth of Rectangle: ");  
    double b = Convert.ToDouble(Console.ReadLine());  
  
    Console.Write("Enter Base of Triangle: ");  
    float ba = Convert.ToSingle(Console.ReadLine());  
  
    Console.Write("Enter Height of Triangle: ");  
    float h = Convert.ToSingle(Console.ReadLine());  
  
    Console.WriteLine("\nArea of Circle   : " + obj.CalculateArea(r));  
    Console.WriteLine("Area of Rectangle : " + obj.CalculateArea(l, b));  
    Console.WriteLine("Area of Triangle : " + obj.CalculateArea(ba, h));  
  
    Console.ReadLine();  
}  
}
```

OUTPUT:

Enter Radius of Circle: 3.14

Enter Length of Rectangle: 6

Enter Breadth of Rectangle: 9

Enter Base of Triangle: 8

Enter Height of Triangle: 5

Area of Circle : 30.959144

Area of Rectangle : 54

Area of Triangle : 20

2. Write a Program to Implement the Compile Time Polymorphism

a. Write a user defined Math class that implements the overloaded method to implement the mathematical functions.

b. Similarly implement the method overloading for Multiply, divide and subtract functionalities

using System;

```
class Math
```

```
{
```

```
    // Add
```

```
    public int Add(int a, int b)
```

```
    {
```

```
        return a + b;
```

```
    }
```

```
public int Add(int a, int b, int c)
{
    return a + b + c;
}
```

// Subtract

```
public int Subtract(int a, int b)
{
    return a - b;
}
```

```
public int Subtract(int a, int b, int c)
{
    return a - b - c;
}
```

// Multiply

```
public int Multiply(int a, int b)
{
    return a * b;
}
```

```
public int Multiply(int a, int b, int c)
{
    return a * b * c;
}
```

```
}
```

```
// Divide
```

```
public int Divide(int a, int b)
```

```
{
```

```
    return a / b;
```

```
}
```

```
public int Divide(int a, int b, int c)
```

```
{
```

```
    return (a / b) / c;
```

```
}
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
{
```

```
    Math obj = new Math();
```

```
    Console.WriteLine("Addition of 10 and 20    : " + obj.Add(10, 20));
```

```
    Console.WriteLine("Addition of 10,20,30    : " + obj.Add(10, 20, 30));
```

```
    Console.WriteLine("Subtraction of 20 and 10 : " + obj.Subtract(20, 10));
```

```
Console.WriteLine("Subtraction of 50,20,10 : " + obj.Subtract(50, 20, 10));
```

```
Console.WriteLine("Multiplication of 5 and 4 : " + obj.Multiply(5, 4));
```

```
Console.WriteLine("Multiplication of 2,3,4 : " + obj.Multiply(2, 3, 4));
```

```
Console.WriteLine("Division of 20 and 5 : " + obj.Divide(20, 5));
```

```
Console.WriteLine("Division of 100,5,2 : " + obj.Divide(100, 5, 2));
```

```
Console.ReadLine();
```

```
}
```

```
}
```

OUTPUT:

Addition of 10 and 20 : 30

Addition of 10,20,30 : 60

Subtraction of 20 and 10 : 10

Subtraction of 50,20,10 : 20

Multiplication of 5 and 4 : 20

Multiplication of 2,3,4 : 24

Division of 20 and 5 : 4

Division of 100,5,2 : 10

Exceptions

1. Write a Program to handle FormatException. if a user enters values other than integer while entering the marks of each subject, catch FormatException. The name of the students and marks in 3 subjects are taken from the user while executing the program.

using System;

class Program

{

static void Main(string[] args)

{

string name;

int sub1, sub2, sub3;

try

{

Console.Write("Enter Student Name: ");

name = Console.ReadLine();

Console.Write("Enter Marks in Subject 1: ");

sub1 = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter Marks in Subject 2: ");

```
sub2 = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter Marks in Subject 3: ");
sub3 = Convert.ToInt32(Console.ReadLine());

int total = sub1 + sub2 + sub3;
double average = total / 3.0;

Console.WriteLine("\nStudent Name : " + name);
Console.WriteLine("Total Marks : " + total);
Console.WriteLine("Average Marks: " + average);
}
catch (FormatException)
{
    Console.WriteLine("\nError: Please enter only integer values
for marks.");
}

Console.ReadKey();
}
}
```

OUTPUT:

Enter Student Name: Rahul

Enter Marks in Subject 1: 85

Enter Marks in Subject 2: abc

Error: Please enter only integer values for marks.

2.In the same Program write a custom exception class called NegativeNumberException if the marks entered are less than zero

using System;

```
// Custom Exception Class
```

```
class NegativeNumberException : Exception
```

```
{
```

```
    public NegativeNumberException(string message) : base(message)
```

```
    {
```

```
    }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

```
        string name;
```

```
        int sub1, sub2, sub3;
```

```
try
{
    Console.Write("Enter Student Name: ");
    name = Console.ReadLine();

    Console.Write("Enter Marks in Subject 1: ");
    sub1 = Convert.ToInt32(Console.ReadLine());

    if (sub1 < 0)
        throw new NegativeNumberException("Marks cannot be negative.");

    Console.Write("Enter Marks in Subject 2: ");
    sub2 = Convert.ToInt32(Console.ReadLine());

    if (sub2 < 0)
        throw new NegativeNumberException("Marks cannot be negative.");

    Console.Write("Enter Marks in Subject 3: ");
    sub3 = Convert.ToInt32(Console.ReadLine());

    if (sub3 < 0)
```

```
        throw new NegativeNumberException("Marks cannot be  
negative.");
```

```
        int total = sub1 + sub2 + sub3;
```

```
        double average = total / 3.0;
```

```
        Console.WriteLine("\nStudent Name : " + name);
```

```
        Console.WriteLine("Total Marks : " + total);
```

```
        Console.WriteLine("Average    : " + average);
```

```
    }
```

```
    catch (FormatException)
```

```
    {
```

```
        Console.WriteLine("\nError: Please enter only integer  
values.");
```

```
    }
```

```
    catch (NegativeNumberException ex)
```

```
    {
```

```
        Console.WriteLine("\nError: " + ex.Message);
```

```
    }
```

```
    Console.ReadKey();
```

```
}
```

```
}
```

OUTPUT:

Enter Student Name: Rahul

Enter Marks in Subject 1: 80

Enter Marks in Subject 2: -20

Error: Marks cannot be negative.

3.Catch the following exceptions for the values entered by the user in the Person class created in previous hands-on

using System;

```
// Custom Exception
```

```
class NegativeNumberException : Exception
```

```
{
```

```
    public NegativeNumberException(string message) : base(message)
```

```
    {
```

```
    }
```

```
}
```

```
class Person
```

```
{
```

```
    public string FirstName;
```

```
    public string LastName;
```

```

public string EmailAddress;

public DateTime DateOfBirth;


    public Person(string firstName, string lastName, string email,
DateTime dob)
    {
        FirstName = firstName;
        LastName = lastName;
        EmailAddress = email;
        DateOfBirth = dob;
    }


    public void Display()
    {
        Console.WriteLine("\nPerson Details");
        Console.WriteLine("-----");
        Console.WriteLine("First Name : " + FirstName);
        Console.WriteLine("Last Name  : " + LastName);
        Console.WriteLine("Email    : " + EmailAddress);
        Console.WriteLine("Date of Birth : " +
DateOfBirth.ToShortDateString());
    }
}

```

```
class Program
{
    static void Main(string[] args)
    {
        try
        {
            Console.Write("Enter First Name: ");
            string fname = Console.ReadLine();

            Console.Write("Enter Last Name: ");
            string lname = Console.ReadLine();

            Console.Write("Enter Email Address: ");
            string email = Console.ReadLine();

            Console.Write("Enter Date of Birth (dd/mm/yyyy): ");
            DateTime dob = Convert.ToDateTime(Console.ReadLine());

            // Check future date
            if (dob > DateTime.Today)
            {
```

```
        throw new NegativeNumberException("Date of Birth cannot  
be a future date.");
```

```
    }
```

```
    Person p = new Person(fname, lname, email, dob);
```

```
    p.Display();
```

```
}
```

```
catch (FormatException)
```

```
{
```

```
    Console.WriteLine("\nError: Invalid date format.");
```

```
}
```

```
catch (NegativeNumberException ex)
```

```
{
```

```
    Console.WriteLine("\nError: " + ex.Message);
```

```
}
```

```
catch (Exception ex)
```

```
{
```

```
    Console.WriteLine("\nError: " + ex.Message);
```

```
}
```

```
        Console.ReadKey();
    }
}
```

OUTPUT:

Enter First Name: Rahul

Enter Last Name: Kumar

Enter Email Address: rahul@gmail.com

Enter Date of Birth (dd/mm/yyyy): abc

Error: Invalid date format.

4.FirstName and LastName should not be Empty and only alphabets should be allowed

using System;

using System.Text.RegularExpressions;

// Custom Exception

class InvalidNameException : Exception

{

public InvalidNameException(string message) : base(message)

{

}

}

```
class Person
{
    public string FirstName;
    public string LastName;
    public string EmailAddress;
    public DateTime DateOfBirth;

    public Person(string firstName, string lastName, string email,
DateTime dob)
    {
        FirstName = firstName;
        LastName = lastName;
        EmailAddress = email;
        DateOfBirth = dob;
    }

    public void Display()
    {
        Console.WriteLine("\nPerson Details");
        Console.WriteLine("-----");
        Console.WriteLine("First Name   : " + FirstName);
        Console.WriteLine("Last Name    : " + LastName);
```

```
        Console.WriteLine("Email      : " + EmailAddress);

        Console.WriteLine("Date of Birth: " +
DateOfBirth.ToShortDateString());
    }
}
```

```
class Program
```

```
{
    static void Main(string[] args)
    {
        try
        {
            Console.Write("Enter First Name: ");
            string fname = Console.ReadLine();

            if (string.IsNullOrEmpty(fname))
                throw new InvalidNameException("First Name should not be
empty.");

            if (!Regex.IsMatch(fname, "^[A-Za-z]+$"))
                throw new InvalidNameException("First Name should
contain only alphabets.");
        }
    }
}
```

```
Console.Write("Enter Last Name: ");
string lname = Console.ReadLine();

if (string.IsNullOrEmpty(lname))
    throw new InvalidNameException("Last Name should not be
empty.");

if (!Regex.IsMatch(lname, "^[A-Za-z]+$"))
    throw new InvalidNameException("Last Name should
contain only alphabets.");

Console.Write("Enter Email Address: ");
string email = Console.ReadLine();

Console.Write("Enter Date of Birth (dd/mm/yyyy): ");
DateTime dob = Convert.ToDateTime(Console.ReadLine());

Person p = new Person(fname, lname, email, dob);

p.Display();
}
catch (FormatException)
{
```

```
        Console.WriteLine("\nError: Invalid Date Format.");
    }
    catch (InvalidNameException ex)
    {
        Console.WriteLine("\nError: " + ex.Message);
    }
    catch (Exception ex)
    {
        Console.WriteLine("\nError: " + ex.Message);
    }

    Console.ReadKey();
}
```

OUTPUT:

Enter First Name: Rahul

Enter Last Name: Kumar

Enter Email Address: rahul@gmail.com

Enter Date of Birth (dd/mm/yyyy): 15/06/2002

Person Details

First Name : Rahul

Last Name : Kumar

Email : rahul@gmail.com

Date of Birth: 15-06-2002

File Handling

1. Create a Console Application to create a file and write contents, save and close the file.

```
using System;
```

```
using System.IO;
```

```
class Program
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

```
        // File path
```

```
        string path = "Student.txt";
```

```
        // Create and write to the file
```

```
        StreamWriter sw = new StreamWriter(path);
```

```
        sw.WriteLine("Student Details");
```

```
        sw.WriteLine("-----");
```

```
        sw.WriteLine("Name : Rahul");
```

```
        sw.WriteLine("Roll No : 101");
```

```
        sw.WriteLine("Course : B.Tech");
```

```
        sw.WriteLine("Marks : 95");
```

```
// Save and close the file  
sw.Close();  
  
Console.WriteLine("File created and data written successfully.");  
  
Console.ReadKey();  
}  
}
```

OUTPUT:

Student Details

Name : Rahul

Roll No : 101

Course : B.Tech

Marks : 95